엔터프라이즈 AI 에이전트

Tool 개발 완전 가이드

이 문서 하나로 LangChain Tool 을 즉시 만들 수 있습니다.

원칙 · 코드 구조 · 완성 예제 · 자동 생성 프롬프트 · 에러 코드 표

제 1 장: Tool 개발 6 대 핵심 원칙

🔑 이 원칙을 모두 지켜야 LLM 이 툴을 올바르게 사용하고, 에이전트 루프가 안정적으로 동작합니다.

원칙	핵심	설명
1. 엄격한 스키마 정의	Pydantic 필수	BaseModel 로 입력 스키마 정의. LLM 이 잘못된 파라미터를 넣는 hallucination 원천 차단.
2. Docstring 4 섹션	구조화 필수	[기능] [선행 조건] [사용 시점] [반환값] — LLM 이 언제 이 툴을 써야 하는지 판단하는 근거.
3. 에러는 JSON 반환	raise 금지	Exception 을 raise 하면 에이전트 루프 전체가 중단됨. 반드시 JSON 에러 문자열로 반환.
4. 선행 조건 강제	코드 레벨 검증	선행 툴 결과 없으면 즉시 MISSING_PREREQUISITE 에러 반환. Planner 하드가드와 이중 방어.
5. 보안 검사	입력값 검증	경로 순회(Path Traversal), 날짜 범위 초과, 권한(ACL) 체크 — 해당 툴에 맞게 적용.
6. 단일 책임	1 툴 = 1 기능	하나의 툴은 하나의 명확한 기능만. 복잡한 로직은 여러 툴로 분리하여 순차 호출.

제 2 장: 표준 코드 구조 상세

2-1. ① Pydantic 스키마 정의

ℹ️ args_schema 패턴이 사내 표준. 단순 함수 인자보다 LLM 파라미터 인식률이 높고 유지보수가 쉽습니다.

```
from pydantic import BaseModel, Field
from typing import Optional

class MyToolInput(BaseModel):
    # 필수 파라미터 — ... 으로 표시
    file_path: str = Field(
        ...,
        description=(
            "분석할 엑셀 파일의 상대 경로. "
            "예: './data/measurement.xlsx'. "
            "절대 경로(/)나 상위 경로(..)는 사용 불가."
        )
    )
    # 선행 툴 결과를 파라미터로 받음
    excel_schema: str = Field(
        ...,
        description=(
            "excel_structure_parser 실행 결과 JSON 문자열. "
            "이 값이 없으면 툴이 즉시 에러를 반환합니다."
        )
    )
    # 선택 파라미터 — Optional + 기본값
    target_column: Optional[str] = Field(
        None,
        description="분석할 컬럼명. None 이면 모든 수치 컬럼 분석."
    )
```

2-2. ② Docstring 4 섹션 구조

⚠️ LLM 은 Docstring 만 보고 이 툴을 언제 써야 하는지 결정합니다. 4 섹션을 반드시 지켜 작성하세요.

```
def my_tool(file_path: str, excel_schema: str,
            target_column: Optional[str] = None) -> str:
    """
    [기능]
    이 툴이 수행하는 작업을 한 문장으로 설명합니다.
    예: 엑셀 파일의 수치 데이터에서 이상값(Outlier)을 감지합니다.

    [선행 조건]
    이 툴을 실행하기 전에 반드시 완료되어야 하는 툴을 명시합니다.
    - excel_structure_parser: 파일 구조 파악 (excel_schema 파라미터)
    - header_detector: 헤더 행 위치 특정
    - datatype_inferrer: 컬럼 타입 추론
    선행 조건이 없는 툴은 "없음"으로 명시합니다.

    [사용 시점]
    사용자가 "이상값 찾아줘", "측정값 검증해줘"라고 요청할 때.
    반드시 위 선행 조건의 툴들이 모두 실행된 이후에만 호출합니다.
```

```
    단독으로 호출하면 MISSING_PREREQUISITE 에러가 반환됩니다.

    [반환값]
    성공: JSON {"status":"success","data":{"outliers":{...},...}}
    실패: JSON {"status":"error","error_code":"...","early_stop":bool,...}
    """
```

2-3. ③ 선행 조건 코드 검증

💡 선행 툴 결과가 없으면 실행하지 말고 즉시 에러를 반환합니다. Planner 하드가드와 이중 방어입니다.

```
    # 선행 조건 검증 — 선행 조건이 없는 툴은 이 블록 생략
    if not excel_schema:
        return json.dumps({
            "status":        "error",
            "error_code":    "MISSING_PREREQUISITE",
            "root_cause":    "missing_prerequisite",
            "early_stop":    False,
            "message":       "excel_schema가 없습니다.",
            "suggested_fix": "excel_structure_parser를 먼저 실행하세요."
        }, ensure_ascii=False)
```

2-4. ④ 보안 검사

⚠️ 툴의 성격에 맞는 보안 검사를 적용합니다. 파일 툴 / MES 쿼리 툴 / 일반 툴 각각 다릅니다.

파일 경로 툴 — Path Traversal 방지

```
    if ".." in file_path or file_path.startswith("/"):
        return json.dumps({
            "status":        "error",
            "error_code":    "PATH_TRAVERSAL",
            "root_cause":    "security_violation",
            "early_stop":    True,
            "message":       "허용되지 않은 경로 접근입니다.",
            "suggested_fix": "상대 경로(./data/...)를 사용하세요."
        }, ensure_ascii=False)
```

MES / DynamoDB 쿼리 툴 — 날짜 범위 제한

```
    from datetime import datetime
    MAX_DAYS = int(os.getenv("MES_MAX_DATE_RANGE_DAYS", 90))

    start = datetime.strptime(start_date, "%Y-%m-%d")
    end   = datetime.strptime(end_date,   "%Y-%m-%d")

    if (end - start).days > MAX_DAYS:
        return json.dumps({
            "status":        "error",
            "error_code":    "DATE_RANGE_EXCEEDED",
            "root_cause":    "date_range_too_large",
            "early_stop":    True,
            "message":       f"최대 {MAX_DAYS}일 범위만 허용됩니다.",
```

```
            "suggested_fix": f"날짜 범위를 {MAX_DAYS}일 이내로 줄여주세요."
        }, ensure_ascii=False)
```

ACL 권한 체크 — DynamoDB 스펙 확인 후 적용

```
    # DynamoDB 테이블 구조 확인 후 acl 모듈 완성 예정
    # if not acl.check(user_id=current_user, resource=query_type):
    #     return json.dumps({
    #         "status":     "error",
    #         "error_code": "PERMISSION_DENIED",
    #         "early_stop": True,
    #         "message":    "해당 데이터 접근 권한이 없습니다.",
    #     }, ensure_ascii=False)
```

2-5. ⑤ 로직 실행 및 반환

ℹ 성공도 JSON 으로 반환합니다. Critic 이 파싱하여 품질을 평가할 수 있어야 합니다.

```
    try:
        # 실제 로직 수행
        result = do_something(file_path, excel_schema)

        # 성공 반환 — 반드시 JSON
        return json.dumps({
            "status": "success",
            "data":   result
        }, ensure_ascii=False)

    except Exception as e:
        # 실패 반환 — raise 절대 금지, 반드시 JSON
        return json.dumps({
            "status":        "error",
            "error_code":    "TOOL_EXECUTION_ERROR",
            "root_cause":    "unexpected_error",
            "early_stop":    False,
            "message":       str(e),
            "suggested_fix": "입력값을 확인하고 재시도하거나 관리자에게 문의하세요."
        }, ensure_ascii=False)
```

제 3 장: 완성 코드 예제

✅ 아래 코드를 복사해서 바로 사용할 수 있습니다. 모든 원칙이 적용된 완성형입니다.

3-1. 엑셀 이상값 감지 툴 (선행 조건 있음)

```
import json, os
from pydantic import BaseModel, Field
from langchain_core.tools import tool
from typing import Optional


class OutlierDetectorInput(BaseModel):
    file_path: str = Field(
        ...,
        description=(
            "분석할 엑셀 파일 상대 경로. 예: './data/measurement.xlsx'. "
            "절대 경로(/)나 상위 경로(..)는 사용 불가."
        )
    )
    excel_schema: str = Field(
        ...,
        description=(
            "excel_structure_parser 실행 결과 JSON 문자열. "
            "반드시 먼저 excel_structure_parser를 실행한 뒤 그 결과를 전달하세요."
        )
    )
    target_column: Optional[str] = Field(
        None,
        description="분석할 컬럼명. None이면 모든 수치 컬럼 분석."
    )


@tool(args_schema=OutlierDetectorInput)
def outlier_detector(file_path: str, excel_schema: str,
                     target_column: Optional[str] = None) -> str:
    """
    [기능]
    엑셀 파일의 수치 데이터에서 이상값(Outlier)을 IQR 방식으로 감지합니다.

    [선행 조건]
    - excel_structure_parser: 파일 구조 파악 결과 (excel_schema 파라미터)
    - header_detector: 헤더 위치 확인 완료
    - datatype_inferrer: 수치 컬럼 타입 추론 완료
    위 세 툴이 모두 실행된 이후에만 호출 가능합니다.

    [사용 시점]
    사용자가 "이상값 찾아줘", "측정값 이상한 거 있어?", "데이터 검증해줘"라고
    요청할 때 사용합니다. 선행 조건 없이 단독 호출하면 에러가 반환됩니다.

    [반환값]
    성공: JSON {"status":"success","data":{"outliers":{컬럼:값목록},...}}
    실패: JSON {"status":"error","error_code":"...","early_stop":bool,...}
    """
```

```
# ① 선행 조건 검증
if not excel_schema:
    return json.dumps({
        "status": "error", "error_code": "MISSING_PREREQUISITE",
        "root_cause": "missing_prerequisite", "early_stop": False,
        "message": "excel_schema가 없습니다.",
        "suggested_fix": "excel_structure_parser를 먼저 실행하세요."
    }, ensure_ascii=False)

# ② 보안 검사
if ".." in file_path or file_path.startswith("/"):
    return json.dumps({
        "status": "error", "error_code": "PATH_TRAVERSAL",
        "root_cause": "security_violation", "early_stop": True,
        "message": "허용되지 않은 경로 접근입니다.",
        "suggested_fix": "상대 경로(./data/...)를 사용하세요."
    }, ensure_ascii=False)

# ③ 로직 실행
try:
    import pandas as pd

    if not os.path.exists(file_path):
        return json.dumps({
            "status": "error", "error_code": "FILE_NOT_FOUND",
            "root_cause": "invalid_path", "early_stop": False,
            "message": f"{file_path} 파일을 찾을 수 없습니다.",
            "suggested_fix": "파일 경로가 올바른지 확인하세요."
        }, ensure_ascii=False)

    df = pd.read_excel(file_path)
    numeric_cols = df.select_dtypes(include="number").columns.tolist()
    if target_column:
        if target_column not in numeric_cols:
            return json.dumps({
                "status": "error", "error_code": "TOOL_EXECUTION_ERROR",
                "root_cause": "invalid_column", "early_stop": False,
                "message": f"{target_column} 컬럼이 없거나 수치형이 아닙니다.",
                "suggested_fix": f"사용 가능한 컬럼: {numeric_cols}"
            }, ensure_ascii=False)
        numeric_cols = [target_column]

    # IQR 이상값 감지
    outliers = {}
    for col in numeric_cols:
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
        iqr = q3 - q1
        mask = (df[col] < q1 - 1.5*iqr) | (df[col] > q3 + 1.5*iqr)
        outliers[col] = df[mask][col].tolist()

    # ④ 성공 반환
    return json.dumps({
        "status": "success",
        "data": {
            "outliers":            outliers,
            "total_outlier_count": sum(len(v) for v in outliers.values()),
```

```
                "analyzed_columns":    numeric_cols,
                "total_rows":          len(df),
            }
        }, ensure_ascii=False)

    except Exception as e:
        # ⑤ 예외 반환 — raise 금지
        return json.dumps({
            "status": "error", "error_code": "TOOL_EXECUTION_ERROR",
            "root_cause": "unexpected_error", "early_stop": False,
            "message": str(e),
            "suggested_fix": "입력값 확인 후 재시도하거나 관리자에게 문의하세요."
        }, ensure_ascii=False)
```

3-2. MES 데이터 조회 툴 (선행 조건 없음, 날짜 제한 있음)

```
import json, os
from datetime import datetime
from pydantic import BaseModel, Field
from langchain_core.tools import tool


class MESQueryInput(BaseModel):
    start_date: str = Field(
        ...,
        description="조회 시작일. 형식: YYYY-MM-DD. 예: 2024-01-01"
    )
    end_date: str = Field(
        ...,
        description="조회 종료일. 형식: YYYY-MM-DD. 최대 90일 범위까지 허용."
    )
    line_id: str = Field(
        ...,
        description="조회할 생산 라인 ID. 예: LINE-01, LINE-02"
    )


@tool(args_schema=MESQueryInput)
def mes_query_tool(start_date: str, end_date: str, line_id: str) -> str:
    """
    [기능]
    사내 MES 시스템에서 특정 라인의 생산/품질 데이터를 조회합니다.

    [선행 조건]
    없음. 이 툴은 MES 파이프라인의 첫 번째 툴입니다.

    [사용 시점]
    사용자가 "생산 현황 보여줘", "불량률 알려줘", "라인 데이터 조회해줘"라고
    요청할 때 사용합니다. 날짜 범위를 반드시 확인하고 호출하세요.
    날짜 범위가 없으면 먼저 사용자에게 확인 후 호출합니다.

    [반환값]
    성공: JSON {"status":"success","data":{"records":[...],"summary":{...}}}
    실패: JSON {"status":"error","error_code":"...","early_stop":bool,...}
    """
```

```
    # ① 날짜 범위 검증
    MAX_DAYS = int(os.getenv("MES_MAX_DATE_RANGE_DAYS", 90))
    try:
        start = datetime.strptime(start_date, "%Y-%m-%d")
        end   = datetime.strptime(end_date,   "%Y-%m-%d")
    except ValueError:
        return json.dumps({
            "status": "error", "error_code": "TOOL_EXECUTION_ERROR",
            "root_cause": "invalid_date_format", "early_stop": False,
            "message": "날짜 형식이 올바르지 않습니다.",
            "suggested_fix": "YYYY-MM-DD 형식을 사용하세요. 예: 2024-01-01"
        }, ensure_ascii=False)

    if (end - start).days > MAX_DAYS:
        return json.dumps({
            "status": "error", "error_code": "DATE_RANGE_EXCEEDED",
            "root_cause": "date_range_too_large", "early_stop": True,
            "message": f"최대 {MAX_DAYS}일 범위만 허용됩니다.",
            "suggested_fix": f"날짜 범위를 {MAX_DAYS}일 이내로 줄여주세요."
        }, ensure_ascii=False)

    # ② 로직 실행
    try:
        # 실제 DynamoDB 쿼리 (스펙 확인 후 구현 예정)
        records = query_dynamodb(line_id, start_date, end_date)

        return json.dumps({
            "status": "success",
            "data": {
                "records":    records,
                "row_count":  len(records),
                "line_id":    line_id,
                "date_range": f"{start_date} ~ {end_date}",
            }
        }, ensure_ascii=False)

    except Exception as e:
        return json.dumps({
            "status": "error", "error_code": "TOOL_EXECUTION_ERROR",
            "root_cause": "unexpected_error", "early_stop": False,
            "message": str(e),
            "suggested_fix": "라인 ID와 날짜 형식을 확인하세요."
        }, ensure_ascii=False)
```

제 4 장: 표준 에러 코드

⚠️ 모든 툴은 아래 표준 에러 코드를 사용합니다. early_stop 이 True 이면 Critic 이 재시도하지 않습니다.

error_code	early_stop	설명 및 처리 방법
MISSING_PREREQUISITE	False	선행 툴 결과 없음. 선행 툴 먼저 실행 후 재시도 가능.
FILE_NOT_FOUND	False	파일/경로 없음. 경로 확인 후 재시도 가능.
INVALID_SCHEMA	False	파일 구조 파악 실패. 다른 시트 또는 Vision 폴백 시도.
TOOL_EXECUTION_ERROR	False	예상치 못한 오류. 입력값 확인 후 재시도 가능.
DATA_CORRUPTED	True	데이터 자체 오염. 재시도해도 동일. 즉시 종료.
FILE_NOT_READABLE	True	파일 손상. 재시도해도 동일. 즉시 종료.
DATE_RANGE_EXCEEDED	True	날짜 범위 90 일 초과. 사용자가 범위 줄여서 재요청.
PERMISSION_DENIED	True	권한 초과. 재시도 없이 즉시 종료.
PATH_TRAVERSAL	True	허용되지 않은 경로. 보안 위반. 즉시 종료.

에러 JSON 구조 (전체 필드)

```
{
    "status":        "error",            // 항상 "error"
    "error_code":    "FILE_NOT_FOUND",   // 위 표의 코드 중 하나
    "root_cause":    "invalid_path",     // Critic 원인 분류에 사용
    "early_stop":    false,              // true 면 Critic 이 즉시 종료
    "message":       "경로를 찾을 수 없습니다.",
    "suggested_fix": "파일 경로가 올바른지 확인하세요."
}
```

성공 JSON 구조

```
{
    "status": "success",
    "data": {
        // 툴마다 다름. Critic 이 파싱하여 품질 평가.
    }
}
```

제 5 장: 툴 자동 생성 프롬프트

🔑 아래 프롬프트를 ChatGPT, Claude, Gemini 등에 붙여넣고 [요구 사항]만 채우면 바로 사용 가능한 코드를 얻을 수 있습니다.

5-1. 프롬프트 (복사용)

```
너는 엔터프라이즈급 AI 에이전트 시스템을 위한 수석 Python 백엔드 엔지니어야.
아래 <개발 규칙>을 완벽히 준수하는 LangChain Custom Tool 코드를 작성해줘.

### <개발 규칙>

1. Pydantic 스키마 필수:
   - 입력 파라미터를 pydantic.BaseModel 상속 클래스로 정의
   - Field(description="...")로 각 파라미터를 LLM이 이해하도록 상세 기술
   - @tool(args_schema=클래스명) 형태로 데코레이터 연결

2. Docstring 4섹션 (반드시 모두 포함):
   [기능]      이 툴이 하는 일을 한 문장으로
   [선행 조건] 이 툴 실행 전 반드시 실행되어야 할 툴. 없으면 "없음" 명시.
   [사용 시점] LLM이 이 툴을 언제 호출해야 하는지
   [반환값]    성공/실패 JSON 구조

3. 선행 조건 코드 검증 (선행 조건이 있는 툴만):
   - 선행 툴 결과 파라미터가 없으면 즉시 MISSING_PREREQUISITE 에러 반환

4. 보안 검사 (해당 항목만 적용):
   - 파일 경로: ".." 또는 "/" 포함 시 → PATH_TRAVERSAL 에러 (early_stop:true)
   - 날짜 범위: 90일 초과 시 → DATE_RANGE_EXCEEDED 에러 (early_stop:true)

5. 에러 반환 규칙 (raise 절대 금지):
   - 모든 에러는 아래 JSON 구조로 반환:
     {"status":"error","error_code":"...","root_cause":"...",
      "early_stop":bool,"message":"...","suggested_fix":"..."}
   - early_stop 기준:
     True  → DATA_CORRUPTED, FILE_NOT_READABLE, PERMISSION_DENIED,
             DATE_RANGE_EXCEEDED, PATH_TRAVERSAL
     False → 그 외 (재시도 가능)

6. 성공 반환:
   - {"status":"success","data":{...}} JSON 구조 사용

7. 타입 힌팅 필수. 반환값은 항상 str.
8. 즉시 복붙 가능한 완성 코드 1개만 제공.

========================================
[요구 사항]

기능 설명: (여기에 입력)
선행 조건: (필요한 선행 툴 결과 파라미터 기술. 없으면 "없음")
보안 검사: (파일경로 / 날짜범위 / ACL / 없음 중 선택)
========================================
```

5-2. 작성 예시

항목	내용
기능 설명	사내 MES DynamoDB 에서 특정 라인의 날짜 범위 불량 데이터 조회. start_date, end_date, line_id 를 입력받아 불량 현황 반환.
선행 조건	없음 (MES 파이프라인 첫 번째 툴)
보안 검사	날짜범위 (90 일 초과 차단)

제 6 장: 툴 완성 전 체크리스트

🚨 코드 작성 후 아래 항목을 모두 확인하세요. 하나라도 빠지면 에이전트가 오동작합니다.

항목	확인 내용
☐ Pydantic 스키마	BaseModel 상속, Field(description=...) 모든 파라미터에 적용, @tool(args_schema=...) 연결
☐ Docstring 4 섹션	[기능] [선행 조건] [사용 시점] [반환값] 모두 존재하는가
☐ 선행 조건 검증	선행 파라미터 없으면 MISSING_PREREQUISITE 반환 (선행 조건 없는 툴은 생략)
☐ 보안 검사	파일툴: Path Traversal, MES 툴: 날짜 범위, 공통: 권한 체크 (해당 항목만)
☐ 에러 JSON 구조	status, error_code, root_cause, early_stop, message, suggested_fix 모두 포함
☐ 성공 JSON 구조	{"status":"success","data":{...}} 형태
☐ raise 없음	try-except 안에서 raise 사용하지 않는가
☐ 반환값 타입 str	모든 반환 경로가 str(JSON 문자열)인가
☐ early_stop 정확성	재시도 불가한 에러에 True, 재시도 가능한 에러에 False 설정